ICC 2026 - IEEE International Conference on Communications | 979-8-3195-4209-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICC59461.2026.11587512

From Monitoring to Prediction: Hands-on Experience on Building a Cloud-Native Network Digital Twin for 5G Core

Lanfranco Zanzi*, Xi Li*, Marco Liebsch*, Fujita Satoshi‡, Datta Kalaswan‡, Sagara Mio‡
*NEC Laboratories Europe, Germany, ‡NEC Corporation, Tokyo, Japan

***Abstract*— The increasing complexity and dynamism of modern communication networks have driven the need for Network Digital Twins (NDTs)—virtual replicas that enable real-time monitoring, analysis, and predictive management of network behavior. However, realizing operational NDTs requires an integrated data pipeline capable of continuous telemetry ingestion, scalable computation, and intelligent decision support. This paper presents a cloud-native monitoring and prediction framework that enables the creation of functional NDT instances for commercial 5G Core performance management. The system fuses real-time telemetry with machine learning–driven analytics to forecast critical performance indicators such as CPU and memory stress, allowing for proactive fault management and network resource control. Built on a modular and containerized architecture and embedded into Amazon Web Service (AWS), the framework supports seamless deployment and interoperability within Kubernetes environments. Experimental validation on stressing network workloads demonstrates its capability to deliver accurate predictions of system degradation and facilitate closed-loop control strategies. The proposed approach opens a practical path toward the realization of self-optimizing, AI-enhanced network digital twins for next-generation autonomous networks.**

I. Introduction

The evolution of mobile communication networks has fundamentally transformed the network architecture, shifting from monolithic, hardware-based systems to cloud-native, service-based architectures (SBA). The 5G Core (5GC) introduces microservices-based network functions (NFs) enabling dynamic scalability, service agility, and multi-vendor interoperability. While this transition enhances flexibility, it also introduces complexity in network operations, orchestration, and lifecycle management [1]. Monitoring, validating, and optimizing such distributed systems in real time is challenging [2]. This has driven the need for a *Network Digital Twin* (NDT)—a high-fidelity virtual replica of the physical network that continuously synchronizes with live network data. The integration of NDTs with 5GC enables simulation-driven management, predictive fault detection, and closed-loop automation [3]. However, most studies on digital twins in telecommunications have focused on theoretical architectures or proprietary simulations running in an isolated environment, lacking a concrete implementation framework for building a live NDT system which continuously synchronizes with the commercial operational network data to enable a real-time closed-loop operation.

To address this gap, this paper builds a live NDT system integrated with a commercial-grade 5G Core software deployed on Amazon Web Services (AWS), and highlights practical hands-on experience when implementing the NDT solution tailored for 5GC operations. This work demonstrates the feasibility of combining containerized 5G NFs, AWS cloud services, and open-source observability tools to realize a scalable and operationally viable digital twin for 5GC.

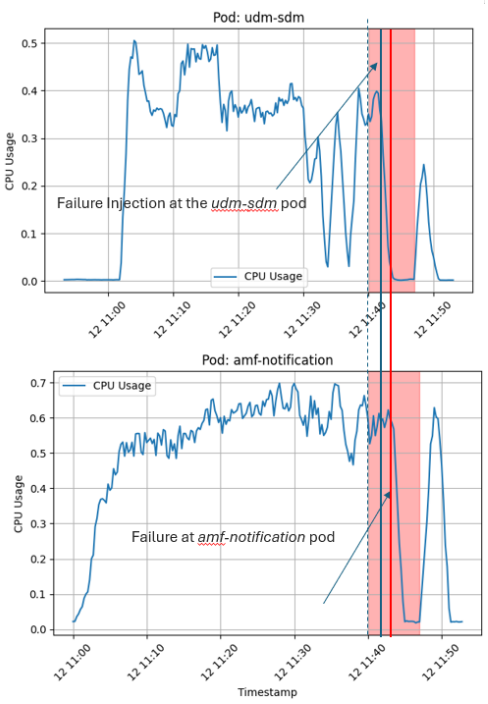


Fig. 1: Example of a cascade failure scenario in a 5G Core deployment, where failure in one network function triggers sequential degradation and restarts in dependent pods.

From Monitoring to Prediction: the increasing scale and complexity of cloud-native network infrastructures, particularly in 5G and beyond, demand new methods for understanding and controlling system behavior under dynamic conditions. Traditional monitoring and alerting mechanisms provide visibility into isolated metrics but without insight on understanding their contextual correlations and lack predictive capabilities needed to manage tightly coupled yet distributed NFs. NDTs address this challenge by maintaining a continuously synchronized virtual replica of the operational network, integrating data-driven analytics, simulation, and machine learning to support real-time reasoning and control [4]. Through such digital replicas, operators can evaluate *"what-if"* scenarios, assess the impact of configuration changes, and anticipate performance degradation before it affects service stability and continuity. An illustrative example is depicted in Fig. 1, which involves cascade failure analysis. A resource exhaustion in one kubernetes pod,

979-8-3195-4209-0/26/$31.00 ©2026 IEEE
Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.

that hosts a micro-service instance of the 5GC, induces successive slowdowns and failure in another dependent pod. Using NDT-based telemetry and predictive models, such interactions can be studied, anticipated, and mitigated proactively, enabling more resilient orchestration policies and self-healing behaviors in large-scale deployments.

Contributions: this paper proposes a cloud-native NDT framework for commercial 5G Core performance management. It consists of a data processing pipeline to pre-process real-time telemetry data collected from the running 5GC system and feed them into an AI/ML–driven analytics to forecast critical performance indicators such as CPU and memory stress per Pod, then allowing for proactive fault management and network resource control. The framework is developed on a modular and containerized architecture and supports seamless deployment and interoperability within Kubernetes environments. Experimental validation was performed by stressing the network workloads, and the results demonstrate the capability of the developed NDT solution able to to deliver accurate predictions of system degradation and facilitate closed-loop control strategies.

The remainder of the paper has the following structure. Section II surveys existing related works and existing approaches to network digital twins. Section III elaborates on the novel architectural designs enabling the NDT concept and detailing the system architecture and implementation methodology. Section IV validates the proposed approach in realistic scenarios. Finally, Section V concludes this paper with insights and future work directions.

II. Related Works

Recent studies have explored NDT frameworks for performance analysis, automated optimization, and closed-loop management of 5G and 6G systems. [5] leverages a set of open source tools to prototype a framework that allows the automated creation and deployment of a network's digital twin, including the steps of data acquisition to build an abstract model of network components, modeling, adaptation, provisioning and inter-connect. The experimental framework has been used to generate a virtualized twin of a 5G network, taking transport, routing and mobile-specific overlay, such as the GTP tunnels on the mobile user plane, into account. Realistic traffic types and patterns are emulated. The framework considers tools for data monitoring and analytics as well as for the creation of data sets for further use by AI models. [6] analyzes various challenges associated with creating and using a NDT, including the collection of data in the physical network and synchronization with the twin, which applies the collected data points to a predictive model. The impact on prediction accuracy is investigated for different data sampling frequency. The authors performed an experimental analysis with a physical private 4G/5G network represented by an experimental system including radio and core network, while the NDT is built with a network emulator that hosts an open source 5G core and radio access network simulator. [7] investigates the use of machine learning (ML) models to predict and optimize resource requirements in Kubernetes clusters, aiming to improve the efficiency of the platform's native Horizontal Pod Autoscaler (HPA). The authors collected performance data from controlled experiments under varying user loads and trained several ML models to forecast CPU and memory utilization for proactive scaling. Their work also highlights the fact that no single ML model perfectly predicted future resource usage, but several models can achieve near-optimal estimations. In [3], the authors present an NDT solution for 5G indoor cellular network that leverages real-time data to predict the UE downlink throughput based on its position. Their approach integrates neural networks with advanced feature selection to model and predict key performance indicators. The results demonstrate that the NDT can accurately replicate and forecast network behavior, enabling continuous monitoring and automated configuration updates in real-world deployments. The work of [8] presents a Mobile Networks Digital Twin (MNDT) architecture designed to replicate an entire 5G Core network and its evolution toward 6G systems using open-source 5G core implementation. The proposed methodology covers the full MNDT lifecycle, from data collection and modeling to real-time synchronization with the physical network, enabling bidirectional communication between digital and physical elements. In [9] the authors present an open and programmable multi-vendor private 5G O-RAN testbed for end-to-end 5G experimentation for slicing and KPI monitoring. A notable contribution is the use of a digital twin-driven ray-tracing framework for optimizing radio unit placement—bridging physical deployment and virtual modeling.

While previous studies have explored predictive modeling and digital twin concepts for network systems, none of the above have addressed the challenges associated with operating within a commercial-grade environment characterized by large-scale deployment, service heterogeneity, and real-time constraints. In contrast, our work introduces a Network Digital Twin that is directly coupled with a live 5G network setup, enabling real-time inference, prediction, and closed-loop feedback to the operational system.

III. 5G Core NDT Framework

A. System Architecture

The implemented system is illustrated in Fig. 2, that has been used for a first proof and analysis of the 5G Core NDT of a live commercial-grade 5G Core system deployed over AWS. It integrates monitoring, data publishing, processing, visualization, analytics and machine learning capabilities into a modular microservices-based architecture deployed within a Kubernetes cluster. This setup enables continuous data ingestion, real-time analytics, and predictive modeling of network behaviour under varying load conditions. In the following, we highlight the main functional service modules of the designed system.

1) 5G Core Network: A commercial-grade 5GC stack is deployed as containerized microservices across AWS Elastic Kubernetes Service (EKS) clusters. The NFs include Access and Mobility Management Function (AMF), Session Management Function (SMF), Unified Data Management (UDM), User Plane Function (UPF), and Policy Control

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.

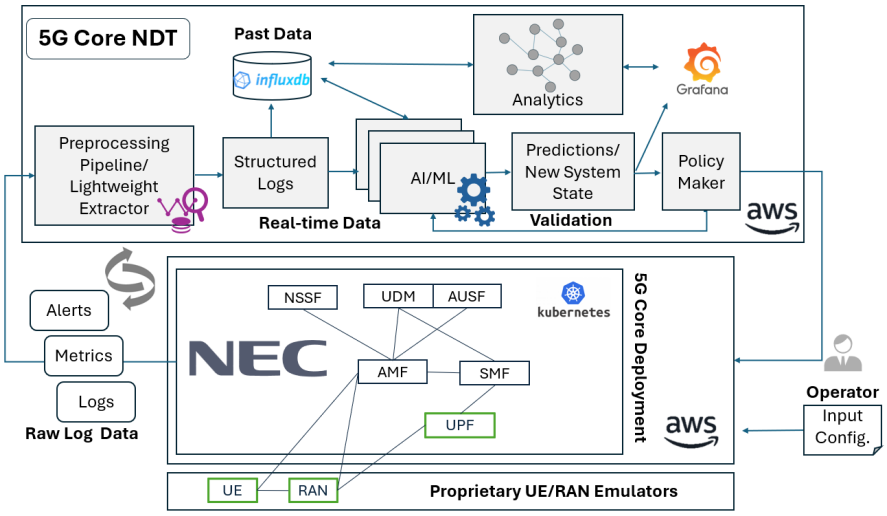


Fig. 2: System architecture Overview

Function (PCF). In addition, we use a proprietary UE/RAN emulator to generate RAN traffic load to the 5GC.

2) Data Storage: The 5GC deployment natively exploits AWS S3 to store data. Nevertheless, to leverage the real-time feature and avoid expensive and slow API calls to AWS service, we setup a dedicated and lightweight InfluxDB instance to store the system metrics. Therefore, from the NDT perspective InfluxDB acts as the central repository for time-series network telemetry data. It is deployed with a dedicated persistent storage of measurement data. The accompanying service exposes the database internally within the cluster to other components that publish or query metrics.

3) Preprocessing Pipeline: The preprocessing service forms the core of the ingestion pipeline. It collects raw logs data, metrics and Key Performance Indicators (KPIs) from the AWS deployed 5G network functions, processes them according to dedicated logic, and structures them before publishing to InfluxDB as time-series. The deployment uses standard resource limits and autoscaling configurations to ensure predictable performance under variable data loads.

4) Visualization: Grafana is deployed as the visualization layer and connected directly to the InfluxDB data source through a predefined datasource ConfigMap. Additional ConfigMaps provide dashboard templates that automatically populate metrics panels upon startup. Grafana is also provisioned with its own persistent volume to store user configurations and dashboards between deployments. The dashboards display data such as CPU and memory utilization, network throughput, and latency measurements, as well as ML-based predictions and performances in terms of accuracy and inference time. Such a tool is useful for a metrics.

5) AI/ML and Prediction: We envision a set of dedicated AI/ML components to provide predictive capabilities across different aspects of network behavior, including resource utilization, performance degradation, and anomaly detection. Rather than relying on a single monolithic tool, the system adopts a modular design in which specialized models operate independently on well-defined tasks, each optimized for its respective data domain and inference objective. This distributed approach enhances scalability, enables targeted model updates, and supports architectural abstraction—allowing the predictive layer to evolve without impacting the rest of the operational stack. Each model can be deployed as an independent microservice within the Kubernetes environment, encapsulated in a container and exposed through a dedicated Flask-based API that serves inference requests and manages updates. Other components, such as monitoring agents or data publisher services, can request predictions via HTTP endpoints, ensuring loose coupling and interoperability. Moreover, each model could independently access real-time information as well as past information stored within the InfluxDB instance. This architecture avoids the need to replicate the entire operational environment externally or maintain full system knowledge in a single model.

6) Policy Maker: In our NDT solution, we envision a Policy-Maker engine to recommend possible reconfigurations or policies to resolve the potential issues or failures which are predicted or detected by the ML models. For instance, in case of NF failure due to overload or congestion on a Pod or a link, the policy-maker can suggest to increase the resource allocation to that pod or interface, or spawn another pod instance to improve load balancing. Before enforcing the policy into the live system, we also use the NDT environment to test the effect of these recommended policies and configurations if they can help resolve the potential issues and which are the performance tradeoffs and/or gains.

B. Data Processing Pipeline

Once the 5GC is deployed and configured based on the input configurations specified by the operators, a set of raw system log data including alerts, metrics and log files are collected. However, one of the key issues is the heterogeneity of these logs that comes in diverse data formats and structures. This diversity complicates the collection and normalization of data necessary for building semantically consistent network representations [8]. A non-exhaustive list

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.

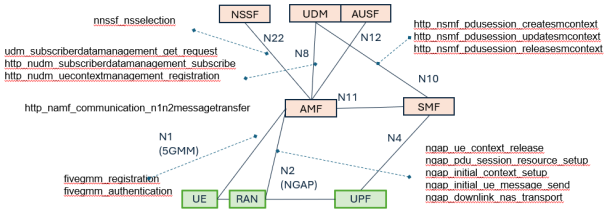


Fig. 3: 5G Core Deployment and subset of selected metrics.

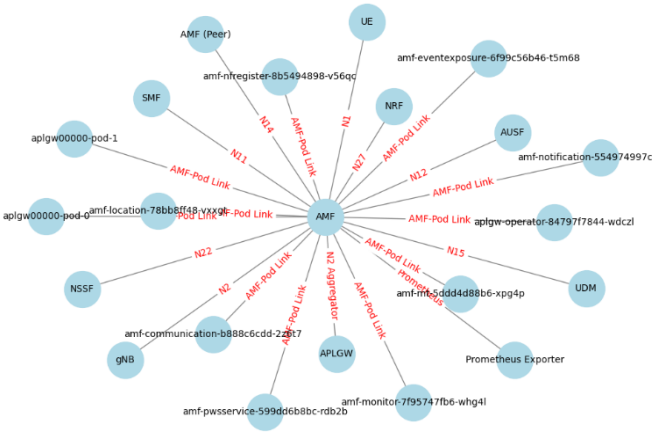


Fig. 4: Relation graph for AMF and its composing pods.

of metrics includes CPU utilization and auxiliary metrics such as memory usage, restarts, I/O, and pod scheduling signals, as well as the set of NF-specific metrics generated by 5G core functionalities over different interfaces, as depicted in Fig.3. Besides, the interactions among pods are characterized by intricate dependencies and multiple interface links. An example of such relationships is depicted in Fig. 4, which highlights the AMF function and its associated pods interconnected with other components of the system. Moreover, the system generates a wide range of metrics that must be jointly evaluated to fully understand its operational behavior. For example, Fig. 5 illustrates the correlations among metrics collected from a single pod composing the AMF service. Such strong and often non-linear relationships between metrics across different pods and functions add another layer of complexity that must be addressed by ML models when learning and predicting system behavior.

In order to preprocess data for ML ingestion, raw log data are normalized to a common data structures $\{\texttt{timestamp, pod, key, value}\}$ and persisted to a time-series database. Streams are synchronized to a fixed sampling period of 15 seconds. Discrete alerts are extracted from event logs and converted to numerical features. To stabilize training, continuous signals are normalized per-pod.

C. ML model for Time-Series Performance Prediction

The NDTs solutions demonstrated promising performances when dealing with a plethora of use-cases, including *"what-if"* analysis based on real-time state, configuration, and behavior. In this work in particular, we focus on per-pod CPU utilization prediction in relation to system traffic loads as one of the main drawbacks of operational 5G networks,

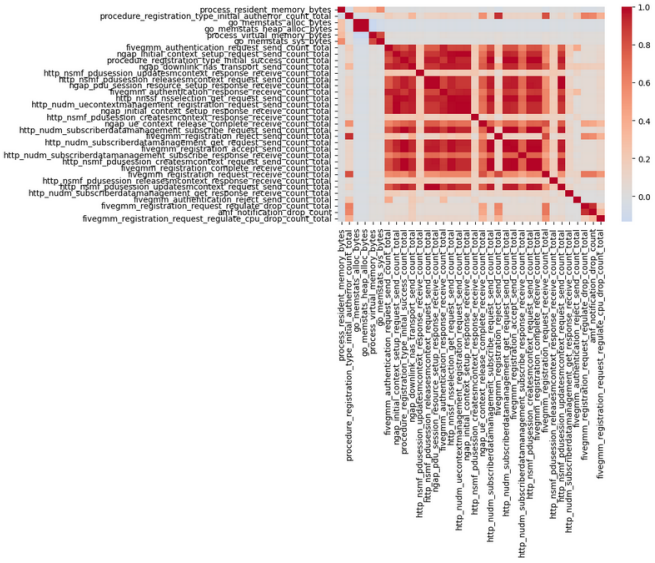


Fig. 5: Feature Correlation Heatmap

which often rely on reactive scaling approaches [10]. In particular, we designed, implemented and trained a predictive engine based on a Transformer encoder architecture tailored for multivariate time-series forecasting of pod-level CPU utilization and alert likelihoods. Input features include normalized telemetry metrics—CPU usage, memory consumption, restart counts, and encoded alert indicators—while targets correspond to future CPU values and/or alert occurrences and its severity per pod. Training sequences are formed with a look-back of $\texttt{WINDOW_SIZE}$ samples and forecast horizon of $\texttt{FUTURE_STEPS}$ samples. Therefore, each input sequence corresponds to a fixed $\texttt{WINDOW_SIZE}$ of past telemetry samples per metric, with a prediction horizon defined by $\texttt{FUTURE_STEPS}$. In addition to system-wide and function-specific metrics, we also compute per-window auxiliary labels to enable both regression and early-warning classification, e.g., CPU $\geq 90\%$ in horizon. Final tensors are scaled (fit on train only), serialized with the scaler parameters, and shuffled in contiguous blocks to preserve local temporal structure during batching.

The model workflow, depicted in Fig. 6, begins with a dense projection layer that maps the input features into a 64-dimensional latent space, followed by a layer normalization stage to stabilize and accelerate training convergence. A multi-head attention mechanism with four attention heads and a key dimension of 64 is then applied to model temporal dependencies and cross-feature interactions, allowing the

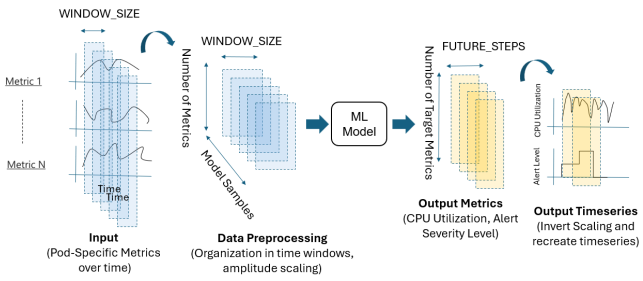
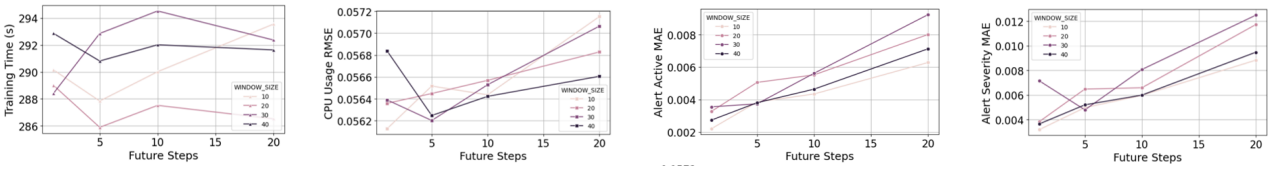


Fig. 6: ML model workflow

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.



(a) Training Time. (b) Root Mean Squared Error of CPU predictions. (c) Mean Absolute Error of Alert predictions. (d) Alert Severity MAE.

Fig. 7: Performance evaluation across different hyperparameter configurations

network to focus selectively on the most relevant parts of the input sequence. After attention encoding, the model includes a nonlinear transformation through a dense layer with 128 ReLU units, complemented by dropout layers with a rate of 0.1 to mitigate overfitting and improve generalization. The resulting representations are flattened and passed to a final dense layer that produces the output corresponding to the required forecast horizon. Positional encodings are added to preserve temporal ordering within each sequence. The model is trained using the Adam optimizer with an initial learning rate of $1 \cdot 10^{-4}$, a batch size of 64, and early stopping based on validation loss (i.e., mean squared error) convergence. The loss functions used are Mean Squared Error (MSE) for CPU utilization and alert regression. Training was performed over 50 epochs using a 70-30 split ratio and an NVIDIA A30 (24GB) GPU.

IV. Performance Evaluation

A. Data Analytics and Telemetry

The proposed framework was deployed in a Kubernetes-based testbed hosting a virtualized 5G Core composed of containerized 5G core network functions (NFs) such as the AMF, SMF, and UPF. Continuous telemetry was collected from multiple layers of the system, including Kubernetes metrics, NF-specific metrics and event logs, as well as alert streams generated by Prometheus agents based on operational deployment settings. All together, more than 60GB of raw log data were collected for the purposes of this work, distributed over more than 500 single features, including CPU utilization, memory consumption, and I/O metrics per pod, along with alert-level indicators such as disk pressure and restart counts. These heterogeneous data sources were aggregated and time-aligned within the InfluxDB time-series database, providing a unified view of system dynamics. This enabled both descriptive analytics—via Grafana dashboards for visual inspection of NF behavior—and predictive analytics through machine learning models. The data set encompassed several days of operational traces, including synthetic stress tests and normal traffic scenarios, producing a rich distribution of temporal correlations among different key performance indicators (KPIs).

B. CPU Predictor

To evaluate the predictive capability of the system, a Transformer-based sequence model was trained to forecast per-pod CPU utilization for short-term horizons. The model takes as input a sliding window of historical telemetry and predicts the next `FUTURE STEPS` samples of CPU load. Its self-attention mechanism enables it to capture long-range dependencies across temporal and cross-pod interactions, which are difficult to model using conventional recurrent or statistical methods. The model was trained using mean-squared-error loss and the Adam optimizer. Fig. 7 summarizes the performances of the Transformer-based solution as a function of different hyperparameter settings. In particular, Fig. 7a first evaluates the model training time, showing a duration that scales nearly linearly with increasing `WINDOW SIZE` and `FUTURE STEPS`, confirming the efficiency of the implemented data pipeline and model architecture. As expected, the number of training samples available decreases for larger `WINDOW SIZE` values since longer input windows reduce the number of non-overlapping sequences that can be generated from a fixed-length dataset. In terms of predictive performance, Fig. 7b depicted the Root Mean Squared Error (RMSE) of CPU utilization forecasts increases with the prediction horizon, as uncertainty accumulates when predicting further into the future. This degradation is moderate for short-term horizons but becomes more pronounced beyond several minutes of lookahead. However, the results indicate that increasing dataset size and applying appropriate scaling can smooth these effects and improve generalization. Similarly, Fig. 7c and Fig. 7d depict the Mean Absolute Error (MAE) for alert prediction and corresponding event severity. These tasks exhibit a rising trend with longer forecast horizons, reflecting the difficulty of accurately modeling discrete event probabilities over extended time intervals. Overall, these results demonstrate that the model maintains stable performance for short to mid-range forecasts and scales effectively across different hyperparameter configurations, supporting its suitability for continuous online prediction in Network Digital Twin environments.

In terms of prediction capabilities, Fig. 8 (left-hand side) illustrates the training and validation loss curves over successive epochs using `WINDOW SIZE` $= 40$ and `FUTURE STEPS`$= 10$. Both losses decrease steadily and converge without overfitting, indicating effective regularization and appropriate model complexity. The validation loss stabilizes slightly above the training curve, which suggests good generalization to unseen data. The smooth decline of both curves also confirms the stability of the learning rate and batch size configuration during optimization. Moreover, Fig. 8 (right-hand side) depicts a representative example of per-pod CPU utilization prediction for an Access and Mobility Function (AMF) micro-service instance within the 5G Core over a full validation. The figure overlays the ground truth CPU usage (solid blue line) with the predicted values (orange

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.

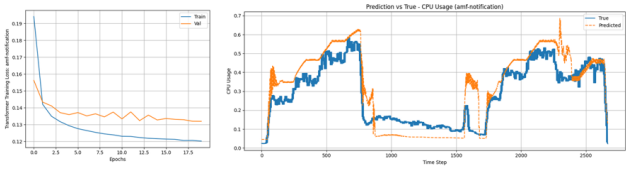


Fig. 8: AMF pod - CPU prediction

line). The model accurately tracks short-term fluctuations and load transients, while slightly smoothing sharp spikes—an expected behavior of sequence models trained with mean-based error metrics. The close alignment between predicted and actual values demonstrates the model's ability to capture temporal dependencies and cross-metric correlations within the pod's telemetry data, which is a key feature to enable 5G Core Digital-Twin solutions.

To assess the real-time feasibility of deploying the predictive model within a Network Digital Twin framework, inference latency was measured across two hardware configurations in a local environment including an NVIDIA A30 GPU (24 GB) and an Intel® Xeon® Gold 6248R CPU @ 3.00 GHz limited to a single core to ease the comparison. The measurements, summarized in Fig. 9, show that CPU and GPU inference times were comparable, with the CPU even outperforming the GPU in certain configurations. This behavior can be attributed to the relatively small model size and limited input dimensionality used in this work. For compact architectures, GPU acceleration often provides little benefit because the parallelization overhead—such as kernel launch latency and data transfer between host and device—can outweigh the computational savings. The CPU, on the other hand, benefits from lower memory-access overhead and efficient cache utilization for smaller computation operations. As a result, the average inference latency remained within the same order of magnitude across both platforms, typically in the range of a few milliseconds per prediction batch. This indicates that the proposed model can be deployed efficiently even on general-purpose CPU nodes, making it well-suited for integration in production-grade Network Digital Twin pipelines without the strict requirement for dedicated accelerators. For larger or more complex models, however, GPU inference is expected to regain its advantage due to higher arithmetic intensity and better throughput at scale.

All together, these findings support the feasibility of deploying AI-driven predictive modules as key enablers of Network Digital Twins, capable of anticipating performance degradation and guiding proactive orchestration decisions.

V. Conclusion

This work presented the design and implementation of a Network Digital Twin for 5G systems capable of real-time monitoring, inference, and prediction of network behavior under realistic operating conditions. Unlike prior approaches that relied primarily on simulation environments or synthetic datasets, our framework is directly integrated with a live network deployment, enabling continuous feedback between the

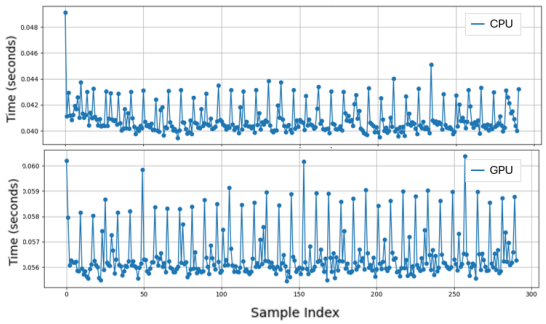


Fig. 9: Average inference time per batch.

physical and digital layers. The developed system supports closed-loop operation, where predictive insights produced by the Digital Twin are fed back into the network to guide resource allocation, performance optimization, and proactive fault mitigation. A central contribution of this work lies in the use of Transformer-based learning models to forecast system behavior across heterogeneous data sources, capturing both short-term variability and long-range temporal dependencies, demonstrating how modern AI architectures can extend the capabilities of Digital Twins from static analytical replicas to adaptive systems that evolve in parallel with their real-world counterparts. Future work will focus on applying Agentic AI to automate log processing, knowledge extraction, and system recovery definition.

References

[1] E. Basikolo *et al.*, "Using Machine Learning to Predict Network Failure in 5G and Beyond," *ITU Journal: ICT Discoveries*, vol. 4, no. 3, 2023.

[2] X. Lin *et al.*, "6G Digital Twin Networks: From Theory to Practice," *IEEE Communications Magazine*, 2023.

[3] O. Stenhammar, S. Sorrentino, F. Chernogorov, R. Blasco, G. Fodor, and C. Fischione, "Real-Time Network Digital Twin: Experimental Setup for Proof of Concept," *IEEE Wireless Communications*, vol. 32, no. 3, pp. 114–121, 2025.

[4] L. Roda-Sanchez, L. Zanzi, X. Li, G. Garí, and X. Costa-Pérez, "Network Digital Twin for 5G-Enabled Mobile Robots," in *IEEE Wireless Communications and Networking Conference*, 2025, pp. 1–6.

[5] M. Sanz Rodrigo, D. Rivera, J. I. Moreno, M. Àlvarez Campana, and D. R. López, "Digital Twins for 5G Networks: A Modeling and Deployment Methodology," *IEEE Access*, vol. 11, pp. 38 112–38 126, 2023.

[6] C. Costa, T. Zhou, and F. Granelli, "A Network Digital Twin of a 5G Private Network: Designing a Proof-of-Concept from Theory to Practice," 10 2025.

[7] A. Rubak and J. Taheri, "Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms," in *IEEE/ACM International Conference on Utility and Cloud Computing*, 2024. [Online]. Available: https://doi.org/10.1145/3603166.3632165

[8] M. Sanz Rodrigo, D. Rivera, J. I. Moreno, M. Àlvarez Campana, and D. R. López, "Digital Twins for 5G Networks: A Modeling and Deployment Methodology," *IEEE Access*, vol. 11, pp. 38 112–38 126, 2023.

[9] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5G: An Open, Programmable, Multi-Vendor, End-to-End, Private 5G O-RAN Testbed With NVIDIA ARC and OpenAirInterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.

[10] T. Subramanya and R. Riggio, "Centralized and Federated Learning for Predictive VNF Autoscaling in Multi-Domain 5G Networks and Beyond," *IEEE Trans. on Network and Service Management*, vol. 18, no. 1, p. 63–78, Mar. 2021. [Online]. Available: https://doi.org/10.1109/TNSM.2021.3050955

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:36:11 UTC from IEEE Xplore. Restrictions apply.